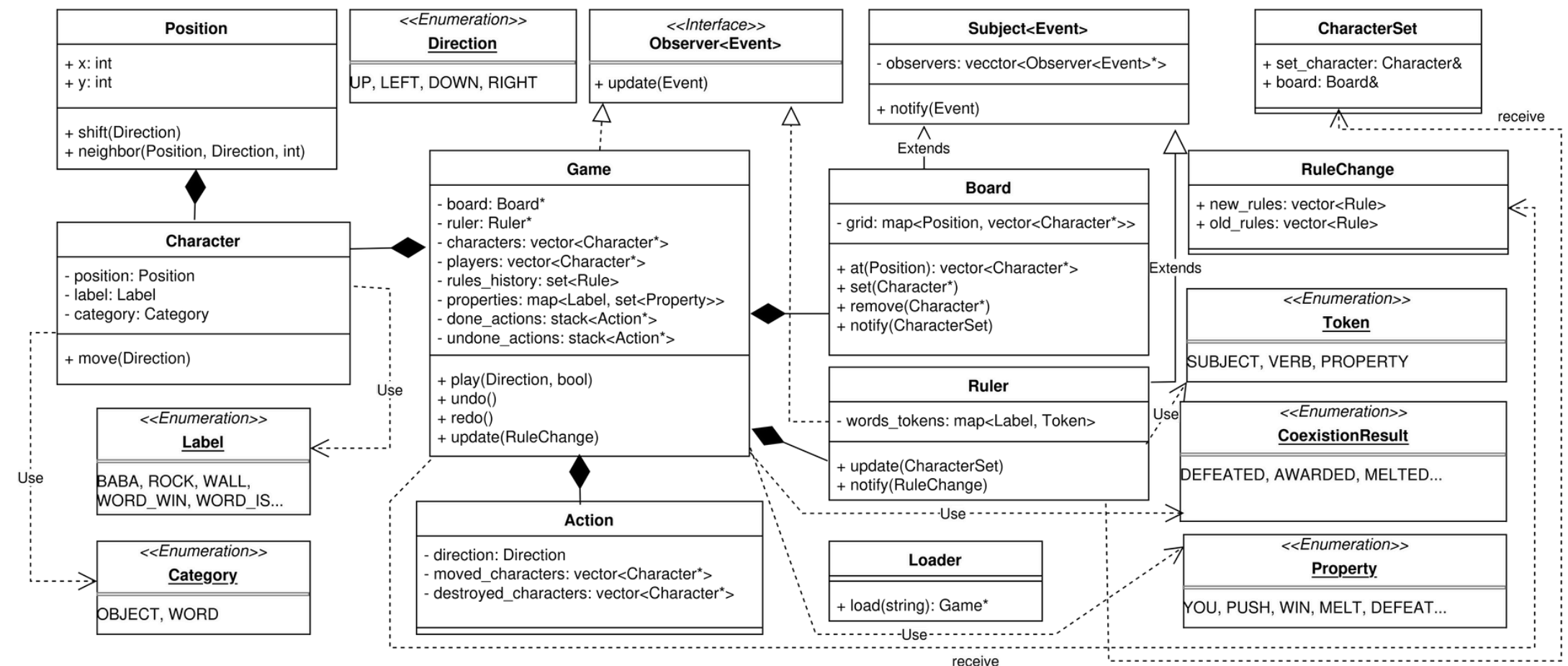


Soutenance du projet C++



UML

- Board notifie à Ruler si un Character a changé
- Ruler vérifie si les règles ont changé puis notifie Game
- Game met à jour les nouvelles règles et/ou éventuellement des anciennes règles
- On a généralisé le pattern afin de pouvoir le réutiliser comme on veut dans tout le projet.



Game

- Game contient des méthodes qui permet d'exécuter des actions.
- L'une des méthodes les plus importantes est le chainage de push, cette fonctionnalité consiste à aller en avant récursivement pour aller chercher une position libre après un bloc poussable.

```
int Game :: pushes_chain_range (Position start, Direction direction) {  
    int range = 0;  
    while (board->in_bounds(start)) {  
        if (has_property(start, Property::STOP)) return -1;  
        if (has_property(start, Property::SHUT) && !has_property(Position::neighbor(start, !direction), Property::OPEN)) return -1;  
        if (has_property(start, Property::PUSH)) {  
            range ++;  
            start.shift(direction);  
        } else break;  
    }  
    if (!board->in_bounds(start)) return -1;  
    return range;  
}
```

Détection de règle

- **Nouvelle règle :** On prend en compte les 6 positions possibles depuis la position notifier par le Board, puis on regarde si les caractères de chacune des positions forme une règle :
SUBJECT VERB (SUBJECT ou PROPERTY)
- **Ancienne règle :** On refait la meme chose mais au lieu de prendre en compte 6 possibilités en doit le faire pour 12

```
void Ruler :: update (const CharacterSet & event) {  
  
    Position event_position = event.set_character.get_position();  
  
    if (event.set_character.get_category() == Category::WORD) {  
  
        vector<LabelT> potential_new_rules = filter_on_rules(  
            filter_on_phrases(  
                get_cells_to_scan(  
                    event.board, get_positions_to_scan_for_rule_creation(event_position)  
                )  
            )  
        );  
  
        vector<LabelT> potential_old_rules = filter_on_rules(  
            filter_on_phrases(  
                get_cells_to_scan(  
                    event.board, get_positions_to_scan_for_rule_destruction(event_position)  
                )  
            )  
        );  
  
        if (!potential_new_rules.empty() || !potential_old_rules.empty()) {  
            notify({potential_new_rules, potential_old_rules});  
        }  
    }  
}
```

Garbage Collector

- Game est propriétaires de tout les pointeurs, donc son destructeur s'en occupe. Notamment pour supprimer tout les Character de Board, les piles qui contiennent des pointeurs d'Action pour le feature undo/redo ainsi que supprimer le Board et Ruler

```
Game :: ~ Game () {  
    for (Character * character: characters) delete character;  
    clear_actions(done_actions);  
    clear_actions(undone_actions);  
    delete ruler;  
    delete board;  
}
```

Loader

- La première ligne initialise le board tel que le premier nombre et la largeur et le nombre suivant est la hauteur
- On initialise après les positions initiales de chaque Character en spécifiant si c'est un Objet (O) ou un Mot (W)
- On initialise aussi les règles à appliquer

```
11 9|
0 0 WORD_BABA W
1 0 WORD_IS W
2 0 WORD_YOU W
8 0 WORD_FLAG W
9 0 WORD_IS W
10 0 WORD_WIN W

0 2 WALL 0
1 2 WALL 0
2 2 WALL 0
3 2 WALL 0
4 2 WALL 0
5 2 WALL 0
6 2 WALL 0
7 2 WALL 0
8 2 WALL 0
9 2 WALL 0
10 2 WALL 0

1 4 BABA 0
5 3 ROCK 0
5 4 ROCK 0
5 5 ROCK 0
9 4 FLAG 0

0 6 WALL 0
1 6 WALL 0
2 6 WALL 0
3 6 WALL 0
4 6 WALL 0
5 6 WALL 0
6 6 WALL 0
7 6 WALL 0
8 6 WALL 0
9 6 WALL 0
10 6 WALL 0

0 7 WORD_WALL W
1 7 WORD_IS W
2 7 WORD_STOP W
8 7 WORD_ROCK W
9 7 WORD_IS W
10 7 WORD_PUSH W

WORD_BABA WORD_IS WORD_YOU
WORD_FLAG WORD_IS WORD_WIN
WORD_WALL WORD_IS WORD_STOP
WORD_ROCK WORD_IS WORD_PUSH
```

Animation

- On n'a malheureusement pas eu assez de temps pour implémenter l'animation, mais on veut quand même montrer que ce n'est pas difficile :
 - On utilise des spritesheet pour récupérer les Character et les mettre dans la view.
 - On récupère la case seulement en connaissant la ligne et la colonne de l'image dans le spritesheet grâce à `sf::IntRect`.
 - Donc il suffit de passer de la colonne initiale à celle d'accoté si l'utilisateur clique sur `sf::Keyboard::Right`, repasser à la case initiale puis rajouter 4 en ligne si l'utilisateur clique sur `sf::Keyboard::Up`, rajouter 8 si c'est `sf::Keyboard::Left` et finalement 12 si c'est `sf::Keyboard::Down`.



**Merci pour votre
attention**

